

LAB 4

1. Videos of lab 4 present here both before and after

[Videos for lab 4 - Google Drive](#)

2. Chat link

<https://chatgpt.com/share/6ac736be-bd90-83ee-b7dd-55605041a6ca>

3. Features / Bug Fixed Snippet

Forked repository link with all the changes - [ridhima-jain45/05_balloon_pop](#)

Commits			
main		All users	All time
Commits on Oct 8, 2026			
Implement round reset feature with 'R' key	Verified	d116505	<>
ridhima-jain45 authored 40 minutes ago			
Implement game over display functionality	Verified	b9d5520	<>
ridhima-jain45 authored 41 minutes ago			
Implement balloon types and lives system	Verified	3969d77	<>
ridhima-jain45 authored 41 minutes ago			
Improve click detection for balloon popping	Verified	38ff843	<>
ridhima-jain45 authored 42 minutes ago			
Refactor Balloon class constructor to add new attributes	Verified	1f87e7b	<>
ridhima-jain45 authored 43 minutes ago			
Commits on Sep 24, 2026			
Initial Balloon Pop starter		c9a5e46	<>
anusha-balamurali committed 2 weeks ago			

balloon.py (fixed)

```
"""
```

```
Balloon: falls from the top of the screen. The player must pop it
before it reaches the bottom. Balloons vary in size - this matters for
how click detection should work.
```

```
"""
```

```
import pygame
```

```
class Balloon:
```

```
    def __init__(
```

```
        self,
```

```
        x,
```

```
        y,
```

```

        radius,
        speed,
        color=(220, 90, 120),
        balloon_type="normal",
        points=10,
    ):
        self.x = x
        self.y = y
        self.radius = radius
        self.speed = speed
        self.color = color
        self.balloon_type = balloon_type
        self.points = points

    def update(self):
        self.y += self.speed

    def is_past_bottom(self, height):
        return self.y - self.radius > height

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius), int(self.y - self.radius),
            self.radius * 2, self.radius * 2,
        )

```

click_detection.py (fixed)

```

"""
click_detection: figures out whether a click landed on a balloon.
"""

def check_pop(balloons, click_pos):
    """

```

```

Returns the balloon that was clicked, or None if the click missed
every balloon.
"""
for balloon in balloons:
    dx = click_pos[0] - balloon.x
    dy = click_pos[1] - balloon.y
    distance_squared = dx * dx + dy * dy

    # Both sides are squared, so clicks anywhere inside the circle,
    # including the edge, are accepted.
    if distance_squared <= balloon.radius * balloon.radius:
        return balloon

return None

```

game_engine.py (fixed)

```

"""
GameEngine: owns all balloons, spawns new ones, and handles clicks.

The engine keeps gameplay state together: balloons, score, lives,
round timer, spawning, popping, misses, and restarting.
"""

import random

from game.balloon import Balloon
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45
POINTS_PER_POP = 10
BONUS_POINTS = 25
PENALTY_POINTS = -10
STARTING_LIVES = 3
ROUND_DURATION = 30.0

```

```

class GameEngine:
    def __init__(self):
        self.reset_round()

    def reset_round(self):
        """Start a completely fresh round."""
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0
        self.lives = STARTING_LIVES
        self.time_remaining = ROUND_DURATION
        self.game_over = False

    def _spawn_balloon(self):
        radius = random.randint(16, 44)
        x = random.randint(radius + 10, WIDTH - radius - 10)
        speed = random.uniform(1.5, 3.0)

        # Normal: +10, Bonus: +25, Penalty: -10.
        roll = random.random()
        if roll < 0.15:
            balloon_type = "bonus"
            color = (60, 210, 90)
            points = BONUS_POINTS
        elif roll < 0.30:
            balloon_type = "penalty"
            color = (80, 100, 230)
            points = PENALTY_POINTS
        else:
            balloon_type = "normal"
            color = (220, 90, 120)
            points = POINTS_PER_POP

        self.balloons.append(
            Balloon(

```

```

        x=x,
        y=-radius,
        radius=radius,
        speed=speed,
        color=color,
        balloon_type=balloon_type,
        points=points,
    )
)

def handle_click(self, pos):
    """Pop a balloon and apply its effect, if the round is active."""
    if self.game_over:
        return

    popped = check_pop(self.balloons, pos)
    if popped is not None:
        self.balloons.remove(popped)
        self.score += popped.points

def _end_round(self):
    self.game_over = True

    # Nothing remains active on the board after the round ends.
    self.balloons.clear()

def update(self, dt=1 / 60):
    """Advance the active round by dt seconds."""
    if self.game_over:
        return

    self.time_remaining = max(0.0, self.time_remaining - dt)
    if self.time_remaining <= 0:
        self._end_round()
        return

    self.frames_until_spawn -= 1

```

```

if self.frames_until_spawn <= 0:
    self._spawn_balloon()
    self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

remaining_balloons = []
for balloon in self.balloons:
    balloon.update()
    if balloon.is_past_bottom(HEIGHT):
        self.lives -= 1
    else:
        remaining_balloons.append(balloon)

self.balloons = remaining_balloons

# A missed balloon is removed here, so it can decrement lives
# only once. Popped balloons never enter this path.
if self.lives <= 0:
    self.lives = 0
    self._end_round()

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(surface, self.balloons)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 40))
    renderer.draw_text(surface, font, f"Time: {self.time_remaining:04.1f}", (10, 70))

    if self.game_over:
        renderer.draw_game_over(surface, font, self.score)

```

renderer.py (fixed)

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (200, 230, 245)
COLOR_TEXT = (30, 30, 30)

def draw_scene(surface, balloons):
    surface.fill(COLOR_BG)
    for b in balloons:
        pygame.draw.circle(surface, b.color, (int(b.x), int(b.y)), b.radius)
        pygame.draw.line(
            surface,
            (120, 120, 120),
            (b.x, b.y + b.radius),
            (b.x, b.y + b.radius + 12),
            2,
        )

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)
```

```

def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)

def draw_game_over(surface, font, score):
    draw_banner(surface, font, "ROUND OVER")

    score_surf = font.render(f"Final Score: {score}", True, COLOR_TEXT)
    score_rect = score_surf.get_rect(
        center=(surface.get_width() // 2, surface.get_height() // 2 + 40)
    )
    surface.blit(score_surf, score_rect)

    restart_surf = font.render("Press R to start a new round", True, COLOR_TEXT)
    restart_rect = restart_surf.get_rect(
        center=(surface.get_width() // 2, surface.get_height() // 2 + 75)
    )
    surface.blit(restart_surf, restart_rect)

```

main.py (fixed)

```

"""
Balloon Pop (Lab Starter)

Run with: python3/python main.py

Click balloons to pop them before they reach the bottom.
Press R after a round ends to start a new round.
"""

import pygame

```

```
from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Balloon Pop")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        dt = clock.tick(60) / 1000.0

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN:
                engine.handle_click(event.pos)
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                if engine.game_over:
                    engine.reset_round()

        engine.update(dt)
        engine.draw(screen, font)

        pygame.display.flip()

    pygame.quit()

if __name__ == "__main__":
    main()
```

Feature / Bug Fixed	What Was Changed
Click Detection	Fixed edge-click detection by comparing squared distance with squared radius.
Balloon Types	Added 3 visually distinct balloon types: Normal (+10) , Bonus (+25) , and Penalty (-10) .
Lives System	Added 3 lives . Missing any balloon costs 1 life .
Countdown Timer	Added a 30-second countdown timer .
Game Over	Game ends when the timer reaches 0 or lives reach 0 .
Restart	Press R after game over to reset the score, lives, timer, balloons, and game state.
Game UI	Added visible Score, Lives, and Time indicators, along with a game-over message.